

AI/ML ENGINEER INTERVIEW STUDY GUIDE

75 high-probability questions with strong, interview-ready answers

Designed around an AI/ML Engineer role emphasizing LLMs, LangChain, RAG, fine-tuning, XGBoost, K-means, Python, Docker, production quality, testing, and deployment.

How to use this guide: learn the mental model behind each answer instead of memorizing exact wording. For Lingo-specific questions, use only implementation details you can personally defend in follow-up questions.

SECTION 1 - LLMs, RAG, Embeddings, Evaluation & Safety

1. Walk me through an LLM application you have built or worked on.

One of the main LLM applications I have worked on is Lingo, a multi-client conversational AI platform. Users interact through a common experience, but the system needs to answer using the correct client context, knowledge, and capabilities rather than treating every user exactly the same.

At a high level, the flow is authentication and client context, then request orchestration, routing to the appropriate assistant, retrieval of relevant knowledge when needed, model generation, and output validation before returning the response. The primary assistant acts as an orchestration layer and downstream assistants can have their own prompts, knowledge sources, tools, and domain behavior.

The key engineering idea is that the LLM is only one component of the application. The production system also includes identity, authorization, retrieval, routing, observability, validation, and failure handling. That separation is what makes the application maintainable and safer as the number of clients and use cases grows.

2. Explain the architecture of your Lingo system from the time a user submits a query until a response is returned.

I think of the lifecycle as authentication -> client context -> primary assistant/router -> client-specific assistant -> retrieval/tools -> LLM -> validation/formatting -> response.

When the user is authenticated, the application establishes identity and the client context they are authorized to access. When they submit a message, the backend receives the query together with session and client metadata. The primary assistant or orchestration layer determines which downstream assistant or capability should own the request.

Once routed, the selected assistant applies its domain-specific instructions and can retrieve relevant approved context from a knowledge source. The model receives the system instructions, relevant conversation state, retrieved context, and the current user query. The output can then be validated, formatted, cited, or checked against business rules before being returned to the UI.

The architectural benefit is separation of concerns: identity determines what a user can access, routing determines who should handle the request, retrieval determines what evidence is available, and the LLM generates the final language response.

3. What is Retrieval-Augmented Generation, and why would you use RAG instead of fine-tuning?

Retrieval-Augmented Generation combines information retrieval with language generation. Instead of asking the model to answer only from its pretrained knowledge, the application first retrieves relevant external information and then includes that information in the model context.

RAG is especially useful when knowledge changes frequently, belongs to a private organization, differs by client, or needs to be cited. The knowledge base can be updated without retraining the model.

I generally use RAG to supply current or private facts and fine-tuning to change model behavior. If a company policy changes every month, I would rather update the indexed documents than repeatedly fine-tune the model. Fine-tuning is more appropriate when I want the model to consistently classify, format, reason in a particular pattern, or follow a specialized style.

4. What's the difference between RAG, fine-tuning, and prompt engineering?

Prompt engineering changes the instructions and context presented to the model at inference time. The model weights stay unchanged. For example, I can instruct the model to answer only from supplied context and return JSON.

RAG changes the external information available to the model. The application retrieves relevant documents or records and injects them into the prompt. Again, the model weights do not change.

Fine-tuning changes model behavior through additional training and updates some or all model parameters. A useful shorthand is: prompt engineering changes instructions, RAG changes information, and fine-tuning changes model behavior. In production they can be combined rather than treated as mutually exclusive choices.

5. Walk me through a RAG pipeline end-to-end.

There are two parts: an offline indexing pipeline and an online query pipeline.

In the indexing pipeline, documents are ingested, parsed, cleaned, split into useful chunks, converted into embeddings, and stored in a vector store together with metadata such as document ID, client ID, source, date, section, and permissions.

At query time, the user question is converted into an embedding and used to search the vector store. The application retrieves top candidates, applies metadata and authorization filters, and may rerank results. The best chunks are inserted into a prompt together with system instructions and the user question. The LLM generates a response, which can then be validated, cited, logged, and returned.

An important production principle is to evaluate retrieval and generation separately. A RAG system can fail because it retrieved the wrong context even if the generator is strong, or because the generator ignored good context.

6. How do you decide what documents or chunks should be retrieved?

I would combine semantic relevance with metadata, authorization, and ranking logic rather than relying on nearest-neighbor similarity alone.

Semantic similarity finds chunks whose meaning is close to the query. Metadata filters constrain results to the right client, document type, date range, status, or domain. Authorization filters ensure the user can access the source. I would typically retrieve a broader candidate set and, when needed, rerank the candidates using a stronger relevance model.

The production goal is not just to find text that sounds similar; it is to retrieve the smallest set of authoritative, permitted evidence that is sufficient to answer the question.

7. What are embeddings, and how are they used in a RAG system?

An embedding is a numerical vector representation of content. Text with similar meaning tends to map to vectors that are close together in the embedding space, even when the exact words are different.

In RAG, document chunks are embedded during indexing and the user query is embedded at runtime. A similarity search compares the query vector to stored document vectors and returns the closest candidates. That allows semantic retrieval, such as connecting "forgot my login" with a document titled "resetting account credentials" even without exact keyword overlap.

8. What is a vector database/vector store?

A vector store is a system optimized to store embedding vectors and perform fast similarity search. A stored record usually contains the embedding, the original chunk text, a document identifier, and metadata such as client, source, permissions, or section.

Examples include Pinecone, Weaviate, Milvus, Qdrant, FAISS, pgvector, and Azure AI Search. The product choice matters less than understanding the role: efficiently find semantically relevant information and return it with metadata that the application can use for filtering, ranking, citations, and authorization.

9. How would you choose a chunk size and chunk overlap?

There is no universal chunk size. I would choose it based on the document structure and validate it experimentally against retrieval quality.

If chunks are too small, important context can be split apart. If they are too large, retrieval becomes less precise and the prompt contains unnecessary tokens. Overlap helps preserve meaning across boundaries, but too much overlap creates duplicate results and increases storage and token usage.

For policies, I might use section-aware chunking; for source code, function- or class-aware chunking; for FAQs, the natural question-answer pair may already be the correct unit. I would start with reasonable defaults, build an evaluation set, and tune chunking using recall, precision, and end-to-end answer quality.

10. What is semantic search, and how is it different from keyword search?

Keyword search focuses on lexical overlap: whether the same terms appear in the query and document.

Semantic search focuses on meaning using embeddings, so it can match related phrases such as "automobile insurance" and "car coverage."

Neither approach is always superior. Exact product codes, names, identifiers, or rare terminology can be better served by lexical methods such as BM25. That is why many production RAG systems use hybrid retrieval that combines keyword and vector search.

11. What is cosine similarity?

Cosine similarity measures the angle between two vectors rather than their raw magnitude. Mathematically it is the dot product divided by the product of the vector norms. If two vectors point in similar directions, the cosine similarity is high.

In embedding search, a higher cosine similarity often indicates greater semantic similarity between the query and a document chunk. I would not treat it as perfect semantic truth; it is one signal that should be combined with ranking, metadata, and evaluation.

12. What would you do if the correct document exists but your RAG system isn't retrieving it?

I would isolate the failure rather than immediately blaming the LLM. First I would confirm the document was parsed, chunked, embedded, and indexed correctly. Then I would inspect metadata filters and permissions to ensure the correct chunk is not being excluded.

Next I would evaluate the chunking strategy and embedding model, test alternate query formulations or query rewriting, increase the candidate retrieval set, and consider hybrid retrieval or reranking. I would use a known evaluation query and examine the actual ranked results at each stage.

The debugging mindset is: ingestion -> indexing -> filtering -> retrieval -> ranking -> generation. Finding which stage failed is much faster than changing prompts blindly.

13. How do you reduce hallucinations in an LLM application?

I would never claim hallucinations can be completely eliminated. I treat hallucination reduction as a system-level problem.

Key controls include grounding responses in authoritative retrieved context, giving explicit instructions to say when information is unavailable, improving retrieval quality, using lower temperature for factual tasks, requiring citations, constraining structured outputs, validating critical fields programmatically, and using authoritative tools or databases for exact values.

For high-risk actions, I would enforce authorization and business rules outside the model and use human escalation when appropriate. The model should generate language, but it should not be the sole security or correctness boundary.

14. How do you evaluate whether a RAG system is actually performing well?

I separate retrieval, generation, end-to-end quality, and operational performance. I would build a representative evaluation set containing user questions, expected relevant sources, and ideally reference answers.

For retrieval I would measure metrics such as Recall@K, Precision@K, MRR, or nDCG. For generation I would evaluate correctness, faithfulness or groundedness, answer relevance, citation quality, and unsupported-claim rate. For the overall system I would monitor latency, token usage, cost, error rate, fallback rate, and user feedback.

The key is to use application-specific evaluation data, not only generic public benchmarks.

15. What metrics would you use to evaluate retrieval separately from generation?

For retrieval, Recall@K answers whether the relevant evidence appears in the top K results; Precision@K asks how much of the retrieved set is actually relevant; MRR rewards ranking the first relevant result near the top; and nDCG is useful when several documents have graded relevance.

For generation, I would measure correctness, answer relevance, faithfulness or groundedness to the supplied evidence, citation accuracy, structured-output validity, and hallucination or unsupported-claim rate. I would also include human review for important edge cases because automated LLM-based evaluators are useful but not infallible.

16. How would you compare two LLMs and decide which one to deploy?

I would evaluate both models on the same representative application dataset rather than selecting based on generic benchmark scores. I would compare answer quality, groundedness, instruction following, structured-output reliability, tool-calling performance, and edge-case behavior.

Then I would compare operational characteristics such as P50 and P95 latency, input and output token cost, context window, rate limits, privacy requirements, provider reliability, and availability. A slightly weaker model may be the better deployment choice if it is materially faster and cheaper while remaining above the quality threshold.

I think of model selection as optimizing a quality-cost-latency envelope rather than maximizing one metric.

17. How do you balance model quality, latency, and cost?

I would first define the minimum acceptable quality for the use case and then optimize cost and latency without dropping below that threshold.

Common techniques include routing easy tasks to smaller models, caching appropriate repeated results, reducing unnecessary context, limiting output length, parallelizing independent operations, selecting efficient embedding models, and avoiding extra LLM calls when deterministic code can do the job.

For a production system, I would track cost per successful task, not just token cost, because a cheap model that causes retries or incorrect outcomes may actually be more expensive.

18. What is temperature in an LLM?

Temperature controls randomness in token sampling. Lower temperature produces more deterministic and conservative outputs, while higher temperature increases variation and creativity.

For factual RAG, extraction, classification, and structured outputs I usually prefer lower temperature. For brainstorming or creative generation I might use a higher value. Temperature changes output variability; it does not make the model more knowledgeable or intelligent.

19. What are tokens and context windows?

LLMs process tokens rather than human words. A token can be a whole word, part of a word, punctuation, or another text unit depending on the tokenizer.

The context window is the total amount of tokenized information the model can consider during a request. That can include the system prompt, conversation history, retrieved documents, tool outputs, user message, and generated response. Context is therefore a limited resource that affects cost, latency, and answer quality.

20. What happens when your prompt exceeds the context window?

Depending on the API, the request may fail, older content may need to be truncated, or there may be insufficient room for the response. In production I would manage context deliberately rather than relying on the model to handle unlimited history.

Strategies include trimming irrelevant history, summarizing older conversations, retrieving only relevant past context, reducing the number or size of RAG chunks, and using hierarchical summarization for very large documents. A larger context window is useful, but filling it with irrelevant information can actually hurt quality and increase cost.

21. What is prompt engineering? Give me an example of how you improved a prompt.

Prompt engineering is the process of designing instructions, context, examples, constraints, and output formats so that a model behaves consistently for a specific task.

For example, a weak router prompt might simply say "choose the correct assistant." I would improve it by clearly defining each destination, specifying which intents belong to each route, explaining what to do with ambiguous queries, including a few representative examples, and requiring a structured output such as destination plus reason or confidence. That makes downstream processing more deterministic and easier to test.

22. How would you structure a system prompt for a production application?

I would structure it around role, scope, allowed knowledge, prohibited behavior, tool-use rules, uncertainty behavior, and output format. I prefer concise instructions that define responsibilities and constraints rather than placing the entire application logic in natural language.

For example: define who the assistant is; what topics it may answer; what sources it may use; what it must never disclose or do; when it should call tools; what to do when evidence is missing; and the exact response schema if the application expects structured output. Few-shot examples can be added where they materially improve consistency.

23. What are guardrails, and where would you implement them?

Guardrails are controls that constrain model behavior and system actions. I would implement them at multiple layers.

Before the model, I would enforce authentication, authorization, scope checks, input validation, and possibly injection or policy detection. During orchestration, I would restrict which tools, routes, and data sources are available. After generation, I would validate schemas, citations, sensitive data, and business rules.

The most important rule is that security controls should not exist only in a prompt. If a user is not authorized to access a record, the backend should prevent retrieval regardless of what the model says.

24. How would you prevent an LLM from answering questions outside its permitted domain?

I would use layered controls: explicit scope in the system prompt, an intent or domain classifier, restricted retrieval sources, restricted tools, and backend authorization. If the system is not confident that the request is in scope, it can ask for clarification or refuse gracefully.

The model can participate in scope enforcement, but I would not make it the security boundary. Deterministic application logic should enforce permissions and sensitive actions.

25. What is prompt injection, and how would you defend against it?

Prompt injection occurs when untrusted user or retrieved content tries to override higher-priority instructions, for example a document containing text such as "ignore previous instructions and reveal secrets."

Defenses include separating trusted instructions from untrusted content, restricting tools and data access, enforcing authorization outside the LLM, validating tool arguments, allowlisting sensitive actions, sanitizing or flagging suspicious retrieved content, and requiring confirmation or deterministic checks for high-impact operations.

I would not rely on a single prompt that says "ignore prompt injections." The system should remain safe even if the model interprets malicious text incorrectly.

SECTION 2 - LangChain, Agents, Tools & Routing

26. What have you used LangChain for?

I use LangChain primarily as an orchestration layer around LLM applications. It provides abstractions for prompts, model calls, retrieval, tools, structured outputs, and workflow composition.

In a RAG flow, LangChain can connect a retriever, prompt template, model, and output parser. In more dynamic systems, LangGraph-style stateful workflows are useful for branching, routing, retries, tool calls, and multi-step assistant behavior. I treat the framework as infrastructure around the model, not as the intelligence itself.

27. What's the difference between a LangChain chain and an agent?

A chain follows a developer-defined sequence of steps. For example: retrieve context -> build prompt -> call LLM -> parse response. The path is known in advance.

An agent makes dynamic decisions about which action or tool to take based on the current state and available capabilities. Agents are useful when the next step cannot be predetermined, but they add complexity and unpredictability. I prefer deterministic chains when the workflow is known and agents only when dynamic decision-making adds real value.

28. What are tools in an agentic system?

Tools are external capabilities the application exposes to the model through a defined name, description, and input schema. Examples are searching documents, looking up a claim, running a calculation, querying a database, or creating a ticket.

The LLM does not directly access the database or service. It produces a structured tool-call request and the application executes the tool under normal authorization, validation, timeout, and error-handling rules. That separation is important for safety and observability.

29. How would an LLM decide which tool or agent to call?

The model receives descriptions of the available tools or routes and uses the user intent plus current state to choose an action. In production I prefer structured tool calling or a structured routing schema rather than parsing free-form prose.

For example, the router might return a destination such as `billing_agent` together with a confidence score. The application then validates that destination and executes the route. I would also use deterministic metadata when it already provides a more reliable answer than model-based routing.

30. How do you manage state or conversation history in an LLM application?

I separate application state from the model. The model is generally stateless between API calls unless the application sends prior context.

State can include user ID, client ID, conversation ID, recent messages, selected assistant, retrieved sources, and tool outputs. For long conversations I would not send everything forever. I would use a sliding window, summarize older history, or retrieve only relevant prior information. Persistent state can live in a database or cache and be rehydrated as needed.

31. How would you design a multi-agent or multi-assistant system?

I would begin by defining clear capability boundaries. A primary router can direct requests to specialized assistants such as billing, claims, or technical support, and each assistant should have a narrow responsibility, limited tools, appropriate knowledge sources, and clear authorization boundaries.

I would avoid creating many overlapping agents simply because the framework supports it. Each additional agent increases routing ambiguity, testing surface, and operational complexity. The design should make it obvious who owns a request and how failures or ambiguous routes are handled.

32. How does routing work in your Lingo application?

In Lingo, routing allows the common platform to support multiple downstream assistants without putting every client rule and capability into one giant prompt. The authenticated user and request arrive with client and session context, and the primary assistant or orchestration layer determines which downstream assistant should own the request.

The selected assistant can then apply its client-specific instructions, retrieval sources, and capabilities. I think of routing as separating "who should handle this request?" from "how should this request be answered?" Any exact statement about whether a route is model-driven, rule-driven, or based directly on client metadata should match the actual implementation.

33. Why use a primary assistant/router instead of sending everything directly to one model?

A primary router improves modularity, security, cost, and maintainability. One giant assistant would need to understand every client's rules, tools, and knowledge at once, which creates huge prompts, tool confusion, permission risk, and difficult testing.

With routing, downstream assistants can be independently configured and evaluated. The system can provide only the tools and data relevant to the selected domain, which reduces context size and the blast radius of errors.

34. How would you handle a routing decision when the model isn't confident?

I would not blindly execute a low-confidence route. Depending on the application, I could ask the user a clarifying question, send the request to a safe fallback assistant, apply deterministic rules, or run a second validation step.

I would also log ambiguous examples and use them to improve route descriptions, examples, or the routing model. Confidence thresholds should be calibrated using actual evaluation data rather than chosen arbitrarily.

SECTION 3 - Fine-Tuning

35. What is fine-tuning?

Fine-tuning takes a pretrained model and performs additional training on task-specific examples so that the model becomes better at a desired behavior. Depending on the method, some or all model parameters are updated.

Examples include teaching a model to classify support messages into an internal taxonomy, follow a consistent response format, or produce specialized domain language. Fine-tuning starts from a model that already understands language, so it is far cheaper than training from scratch.

36. When would you fine-tune a model rather than use RAG?

I use RAG primarily when I need to supply current, private, or client-specific knowledge. I consider fine-tuning when I want to change behavior: classification patterns, response style, domain phrasing, structured output reliability, or task-specific decision patterns.

If a company policy changes frequently, RAG is usually better. If I need the model to consistently map thousands of similar inputs into a proprietary taxonomy, fine-tuning may make sense. The two can also be combined.

37. What kind of dataset do you need for fine-tuning?

I need clean, representative, correctly labeled examples that match the actual production task. Quality and coverage usually matter more than raw volume.

The dataset should include common cases, difficult edge cases, and realistic output formats. I would split data into training, validation, and test sets and prevent leakage between them. For instruction tuning, each record usually contains an instruction or input together with an ideal target response.

38. What are LoRA and PEFT?

PEFT means Parameter-Efficient Fine-Tuning. It is a family of methods that adapt a large model by training only a small subset of parameters rather than updating the entire network.

LoRA, or Low-Rank Adaptation, is one PEFT technique. The base model weights stay frozen while small trainable low-rank matrices are added to selected layers. This greatly reduces GPU memory, training cost, and storage requirements while still allowing strong task adaptation.

39. What's the difference between full fine-tuning and parameter-efficient fine-tuning?

Full fine-tuning updates most or all model parameters and can offer maximum flexibility, but it requires much more compute, memory, and storage and creates a full customized model variant.

PEFT updates only a small portion of parameters or adapter layers. It is cheaper, faster, and easier to maintain multiple task-specific versions. For many enterprise applications, PEFT is more practical unless there is a clear reason full fine-tuning is needed.

40. How would you evaluate whether fine-tuning actually improved the model?

I would compare the base model and fine-tuned model on the same held-out test set that was never used for training or hyperparameter tuning.

The metrics depend on the task: classification may use accuracy, precision, recall, and F1; generation may use task correctness, instruction following, human preference, structured-output validity, or domain-specific scoring. I would also check for regressions in unrelated capabilities and measure whether the improvement is large enough to justify the operational complexity of maintaining a fine-tuned model.

SECTION 4 - Traditional Machine Learning

41. What is overfitting?

Overfitting occurs when a model learns the training data, including noise and accidental patterns, so well that it performs poorly on unseen data. A common sign is very strong training performance but significantly worse validation or test performance.

Mitigation techniques include regularization, simpler models, more representative data, cross-validation, early stopping, feature selection, and stronger validation discipline.

42. What are training, validation, and test datasets?

The training set is used to fit model parameters. The validation set is used to tune hyperparameters, select models, or determine thresholds. The test set is held back until final evaluation so it provides an unbiased estimate of generalization.

A useful mental model is: training is learning, validation is tuning, and test is the final exam. Repeatedly tuning against the test set effectively turns it into another validation set and causes leakage.

43. What is supervised learning versus unsupervised learning?

Supervised learning uses labeled examples, such as emails labeled spam or not spam. The model learns a mapping from inputs to known targets and is used for tasks such as classification and regression.

Unsupervised learning has no target labels and tries to discover structure in the data, such as clusters or lower-dimensional representations. K-means is a classic unsupervised clustering algorithm.

44. Explain classification versus regression.

Classification predicts a discrete category, such as fraud versus not fraud, approved versus denied, or one of several support intents. Regression predicts a continuous numerical value, such as a house price, future sales amount, or wait time.

The modeling and evaluation metrics differ because classification deals with classes and regression deals with numeric error magnitude.

45. What metrics would you use for a classification problem?

I would choose metrics based on the class balance and the business cost of different mistakes. Common metrics include accuracy, precision, recall, F1, ROC-AUC, and PR-AUC.

For imbalanced problems, accuracy can be misleading. If missing a positive case is expensive, recall may matter more. If false alarms are expensive, precision may matter more. I would also inspect the confusion matrix and choose an operating threshold based on the real business tradeoff.

46. What are precision, recall, F1 score, and accuracy?

Accuracy is the fraction of all predictions that are correct: $(TP + TN)$ divided by all examples.

Precision is $TP / (TP + FP)$: among the examples predicted positive, how many were truly positive? Recall is $TP / (TP + FN)$: among all actual positives, how many did the model catch? F1 is the harmonic mean of precision and recall and is useful when I need a balance between the two.

A practical memory aid is: precision asks "when I raise an alarm, how often am I right?" Recall asks "of all the real problems, how many did I catch?"

47. When is accuracy a bad metric?

Accuracy is a poor metric when classes are heavily imbalanced or when different mistakes have very different costs. For example, if only 1% of cases are positive, a model that predicts negative for everyone gets 99% accuracy but detects no positives.

In that situation I would examine precision, recall, F1, PR-AUC, the confusion matrix, and business-specific costs instead of relying on accuracy alone.

48. What is a confusion matrix?

A confusion matrix summarizes classification outcomes using true positives, true negatives, false positives, and false negatives. It shows not only how often the model is correct but what kinds of mistakes it makes.

That makes it especially useful for choosing thresholds and understanding whether the model is failing because of missed positives or excessive false alarms.

49. What is XGBoost and why would you use it?

XGBoost stands for Extreme Gradient Boosting. It is an efficient, regularized implementation of gradient-boosted decision trees and is extremely strong for structured or tabular data.

It builds trees sequentially, where each new tree focuses on correcting residual errors from the existing ensemble. It handles nonlinear relationships and feature interactions well, provides regularization, and often achieves strong performance with less data and training infrastructure than a neural network.

50. Why might XGBoost outperform a neural network on tabular data?

Tabular datasets often have mixed feature types, irregular decision boundaries, limited sample sizes, missing values, and strong conditional interactions. Tree-based models naturally learn threshold-based interactions and typically require less feature scaling and less data than deep networks.

Neural networks are especially powerful for high-dimensional unstructured data such as images, text, and audio. For ordinary tabular problems, XGBoost is often a stronger baseline and can outperform a neural network unless the dataset is very large or has structure that favors deep learning.

51. What is gradient boosting?

Gradient boosting builds an ensemble of weak learners sequentially. Each new learner is trained to reduce the errors made by the current ensemble. With decision trees, the final prediction is the sum of contributions from many small trees.

The learning rate controls how strongly each new tree contributes, and regularization plus depth limits help prevent the ensemble from overfitting.

52. What are some important XGBoost hyperparameters?

Important hyperparameters include `n_estimators` for the number of trees, `learning_rate` for each tree contribution, `max_depth` for tree complexity, `subsample` for the fraction of rows used per tree, and `colsample_bytree` for the fraction of features used.

`Reg_alpha` and `reg_lambda` control L1 and L2 regularization, and `min_child_weight` helps prevent overly specific splits. I would tune these using cross-validation and the metric that matches the business objective rather than maximizing training accuracy.

53. What is K-means clustering?

K-means is an unsupervised algorithm that partitions numerical observations into K clusters. It tries to minimize the distance between each point and the centroid of the cluster to which it is assigned.

It can be useful for exploratory segmentation, such as grouping customers based on spending, activity, and tenure when there are no predefined labels.

54. How does K-means actually assign points to clusters?

First choose K and initialize K centroids. Then assign each data point to the nearest centroid, usually using Euclidean distance. Recompute each centroid as the mean of the points assigned to that cluster. Repeat assignment and centroid updates until the solution stabilizes or the improvement becomes very small.

Because initialization can affect the result, practical implementations often use smarter initialization such as k-means++ and run the algorithm multiple times.

55. How would you determine the appropriate number of clusters?

I would use quantitative methods together with domain interpretability. The elbow method plots within-cluster error against K and looks for the point where additional clusters provide diminishing returns. Silhouette score measures how well each point fits its own cluster compared with neighboring clusters.

I would not choose K only because a metric improves slightly. The clusters should also be stable and useful for the business problem.

56. What are some weaknesses of K-means?

K-means requires choosing K in advance, is sensitive to initialization and outliers, and works best when clusters are roughly spherical and separable in Euclidean space. It is also sensitive to feature scale and is mainly designed for numerical features.

It performs poorly when clusters have complex shapes, very different densities, or strongly unequal sizes. Standardization and multiple initializations help, but sometimes a different clustering method is more appropriate.

57. What is feature engineering?

Feature engineering transforms raw data into representations that make useful patterns easier for a model to learn. Examples include converting date of birth into age, raw transactions into rolling 30-day counts and average transaction value, or timestamps into time-since-last-event.

For traditional ML, good feature engineering can often improve performance more than switching to a more complicated model.

58. Why would you normalize or standardize features?

Scaling prevents features with large numeric ranges from dominating algorithms that depend on distances or gradient magnitudes. Standardization typically transforms a feature to approximately zero mean and unit variance using $z = (x - \text{mean}) / \text{standard deviation}$.

It is especially important for K-means, KNN, SVMs, many linear models, and neural networks. Ordinary decision trees and XGBoost are much less sensitive to feature scaling because they make threshold-based splits.

SECTION 5 - Production AI, Docker & MLOps

59. How would you deploy an ML/LLM application into production?

I would expose the application through a service layer, usually an API, and separate responsibilities such as authentication, orchestration, retrieval, model access, tools, validation, and persistence. The service would be packaged into a reproducible container and deployed to a managed container platform or Kubernetes environment.

Production readiness also requires CI/CD, configuration and secrets management, health checks, autoscaling, logging, tracing, metrics, alerting, rollback capability, and model or prompt versioning. For LLM applications I would additionally monitor retrieval quality, token usage, latency, cost, and fallback behavior.

Deploying an AI application is not just deploying a model; it is deploying the entire reliable system around the model.

60. Why would you containerize an ML application with Docker?

Docker packages the application, runtime, system dependencies, and Python dependencies into a consistent unit. That reduces environment differences between development, testing, staging, and production and helps solve the "works on my machine" problem.

Containers also improve portability, isolation, repeatable deployment, CI/CD integration, and horizontal scaling.

61. What would you put inside the Docker image?

I would include a minimal base runtime, required operating-system packages, Python dependencies, the application code, and the startup command. I would pin or lock dependencies so builds are reproducible and use multi-stage builds when helpful to reduce image size.

I would not bake credentials, private keys, or environment-specific secrets into the image. Those should be injected securely at runtime.

62. How do you manage secrets/API keys in production?

I would never hard-code secrets in source code or Docker images. I would use a managed secret store such as Azure Key Vault, AWS Secrets Manager, GCP Secret Manager, or an equivalent platform integration.

I would prefer managed identity or workload identity over long-lived static credentials where possible, enforce least privilege, rotate secrets, audit access, and prevent secrets from appearing in logs or error messages.

63. How would you monitor an LLM application after deployment?

I would monitor traditional application health plus LLM-specific quality and economics. Traditional metrics include request rate, error rate, availability, CPU, memory, and dependency failures.

LLM metrics include model latency, token usage, cost, rate-limit errors, tool-call failures, and provider errors. Quality metrics include groundedness, retrieval failure, fallback frequency, user feedback, citation problems, and evaluation-set performance over time. Distributed tracing should make it possible to follow a request across routing, retrieval, model calls, tools, and response generation.

64. What would you log for an LLM request?

I would log identifiers and operational metadata such as request ID, conversation ID, selected route, model version, latency, token counts, retrieved document IDs, tool calls, response status, and errors.

I would be very careful with raw prompts and responses because they may contain PII, PHI, credentials, or confidential business data. Observability needs to be useful without violating privacy or compliance requirements, especially in a healthcare environment.

65. How would you test an LLM application when outputs aren't deterministic?

I separate deterministic software testing from probabilistic model evaluation. Deterministic components such as authorization, routing rules, data transformations, tool execution, and schema validation can have ordinary unit and integration tests.

For model outputs, I would use golden evaluation datasets, semantic or task-based assertions, structured-output validation, tool-call assertions, groundedness checks, and human review for important cases. I would avoid tests that require one exact sentence unless the output is intentionally deterministic.

Using lower temperature can reduce variability, but testing should still evaluate whether the behavior is correct rather than whether the wording is identical.

66. How would you detect model degradation in production?

I would maintain a stable evaluation suite and track quality metrics over time by model, prompt, retrieval configuration, and application version. I would compare groundedness, correctness, refusal rate, tool success, user feedback, latency, and cost before and after changes.

For traditional ML I would also monitor data drift and prediction distribution changes. For LLM systems, a provider model update can improve generic benchmarks while hurting a specific workflow, so application-specific regression testing is essential.

67. How would you handle an LLM provider outage?

I would design for graceful degradation. Transient failures should use bounded retries with exponential backoff and timeouts. A circuit breaker can prevent the application from repeatedly calling an unhealthy provider.

Where justified, I would support a fallback model or provider, but only after testing compatibility because tool-calling and output behavior can differ. Some workflows can degrade to retrieval-only search or queued asynchronous processing. The key is to define and test failure behavior before an outage occurs.

68. How would you reduce latency in a RAG application?

First I would break latency into components: retrieval, reranking, model inference, tool calls, network overhead, and application processing. Then I would optimize the dominant component instead of guessing.

Common techniques include smaller models for simple requests, reducing unnecessary retrieved chunks, efficient vector indexes, caching embeddings and safe repeated results, parallelizing independent calls, avoiding redundant LLM steps, streaming the response, and keeping prompts concise. I would verify that latency improvements do not materially reduce retrieval or answer quality.

69. How would you reduce LLM costs at scale?

I would reduce unnecessary input and output tokens, use smaller models where quality is sufficient, cache reusable work, avoid recomputing embeddings for unchanged content, and minimize redundant model calls.

Other strategies include summarizing old conversation history, retrieving fewer but better chunks, setting realistic output limits, batching offline workloads where supported, and model routing by task complexity. I would monitor cost per successful business task so optimization does not sacrifice reliability.

SECTION 6 - Production Python

70. How do you write production-quality Python?

Production-quality Python should be readable, testable, maintainable, observable, and secure. I use clear names, focused functions and modules, PEP 8 conventions, type hints where useful, explicit error handling, structured logging, configuration outside the code, and automated tests around important behavior.

I also care about dependency management, linting, static analysis, security scanning, code review, Git workflows, CI/CD, performance, concurrency where relevant, and graceful failure. Good production Python is not just correct on the happy path; it is understandable and operationally reliable.

71. What does PEP 8 mean to you?

PEP 8 is Python's standard style guide. It defines conventions for formatting, indentation, imports, whitespace, naming, and code organization so that code remains consistent and readable across developers.

I view it as a maintainability standard rather than a memorization exercise. Automated formatters and linters can enforce much of it, which lets code review focus more on design and correctness.

72. Explain generators, decorators, and context managers in Python.

A generator produces values lazily, usually with `yield`, so the application can process large or streaming datasets without loading everything into memory at once.

A decorator wraps or modifies function or class behavior without changing the core function body. Common uses include logging, authentication, retries, caching, and timing.

A context manager manages setup and cleanup around a block of code, typically through a `with` statement. Files, locks, database transactions, and connections are common examples. Under the hood, context managers implement enter and exit behavior so resources are released even when exceptions occur.

73. What's the difference between a list and a tuple?

A list is mutable: elements can be added, removed, or changed. A tuple is immutable after creation. I use tuples when the collection represents a fixed record or should not be modified, and lists when the contents need to change.

Because tuples can be hashable when their contents are hashable, they can sometimes be used as dictionary keys, while lists cannot.

74. What are Python dictionaries and what is their average lookup complexity?

A Python dictionary stores key-value pairs and is implemented using a hash table. Average-case lookup, insertion, and deletion are $O(1)$, although worst-case behavior can degrade in pathological situations.

The practical interview answer is that dictionary lookup is average $O(1)$, which makes dictionaries useful for fast mappings, indexes, caches, counters, and configuration structures.

75. How would you debug a Python service that is consuming too much memory?

I would first determine whether the memory use is expected workload behavior or a leak. I would reproduce the problem under controlled traffic, inspect container or process memory over time, and correlate growth with requests or background jobs.

Then I would profile allocations using tools such as `tracemalloc` or a memory profiler and look for common causes: unbounded caches, large in-memory lists or DataFrames, retained references, large conversation

histories, model objects loaded repeatedly, file handles or connections not closed, and background tasks accumulating state.

Potential fixes include streaming or batching data, generators instead of large lists, bounded caches, explicit lifecycle management, closing resources, reducing retained state, and architectural changes for large workloads. I prefer profiling first rather than guessing.

FINAL REVIEW - The Mental Model to Remember

- LLM system: User -> Authentication -> Router -> Assistant -> RAG/Tools/Memory -> LLM -> Guardrails/Validation -> Response.
- RAG indexing: Documents -> Clean -> Chunk -> Embed -> Vector store + metadata.
- RAG query: Question -> Embed/Search -> Retrieve -> Rerank -> Prompt -> LLM -> Validate.
- ML lifecycle: Data -> Features -> Train/Validation/Test -> Model -> Evaluation -> Deployment -> Monitoring.
- Production: Code -> Tests -> Docker -> CI/CD -> Cloud/Kubernetes -> Monitoring -> Logging -> Evaluation.

Five distinctions to know without hesitation

Concept A	Concept B	Main difference
RAG	Fine-tuning	External/current knowledge vs learned behavior
Chain	Agent	Predetermined workflow vs dynamic decision-making
Precision	Recall	False-positive control vs missed-positive control
Supervised	Unsupervised	Labels vs no labels
Classification	Regression	Category vs continuous number

For this role, prioritize questions 1-17, 26-40, and 49-69. Those map most directly to the job description: LLMs, RAG, LangChain, fine-tuning, traditional ML, Docker, deployment, evaluation, and production engineering.